

Chương 6: Cây Quyết định (Decision Trees)

Giảng viên

Đại học Đông Á

August 11, 2026

Mục tiêu của Chương

- Hiểu rõ thuật toán **Cây Quyết định (Decision Trees)** và khả năng của nó.
- Cách một mô hình cây đưa ra dự đoán (Phân loại và Hồi quy).
- Thuật toán **CART** (Classification and Regression Tree) dùng để huấn luyện.
- Tìm hiểu về độ vẩn đục: **Gini Impurity** và **Entropy**.
- Các kỹ thuật **Chính quy hóa (Regularization)** bằng siêu tham số.
- Điểm yếu của Cây Quyết định: Phương sai cao (High Variance) và nhạy cảm với hướng trục.

Mục lục

1. Khái quát về Cây Quyết định
2. Huấn luyện và Trục quan hóa
3. Đưa ra dự đoán (Making Predictions)
4. Thuật toán Huấn luyện CART
5. Chính quy hóa Cây (Regularization)
6. Hồi quy bằng Cây quyết định
7. Các Điểm yếu của Cây Quyết Định
8. Tổng kết

1.1 Sức mạnh của Decision Trees

- Cây Quyết định là một trong những thuật toán Học máy đa năng nhất.
- Có thể thực hiện cả **Phân loại (Classification)**, **Hồi quy (Regression)**, và học nhiều đầu ra (Multioutput).
- Khả năng khớp với các tập dữ liệu cực kỳ phức tạp (Non-linear datasets).
- Là thành phần cơ bản (Building block) của các thuật toán cực mạnh như **Random Forest** và **Gradient Boosting**.

1.2 Mô hình Hộp Trắng (White Box Model)

- Điểm sáng giá nhất của Cây Quyết định là tính **dễ diễn giải (Interpretability)**.
- Nó được gọi là mô hình Hộp Trắng (White Box) vì ta có thể theo dõi chính xác cách mô hình đi đến kết luận.
- Trái ngược với mô hình Hộp Đen (Black Box) như Mạng nơ-ron hay Random Forest, nơi rất khó giải thích tại sao mô hình lại đưa ra dự đoán đó.
- Được sử dụng rộng rãi trong các lĩnh vực Y tế, Tài chính, nơi cần giải trình quyết định.

2.1 Code: Huấn luyện cây quyết định

- Sử dụng bộ dữ liệu kinh điển Iris (Hoa diên vĩ).
- Ta sẽ chỉ sử dụng 2 đặc trưng: petal length (chiều dài cánh hoa) và petal width (chiều rộng).

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

# Tải tập dữ liệu Iris
iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target

# Huấn luyện mô hình Cây Quyết định
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
```

2.2 Code: Xuất biểu đồ cây (Graphviz)

- Scikit-Learn cung cấp phương thức `export_graphviz` để vẽ cây thành file `.dot`.
- Sau đó dùng Graphviz để chuyển `.dot` thành ảnh PNG hoặc PDF.

```
from sklearn.tree import export_graphviz

export_graphviz(
    tree_clf,
    out_file="iris_tree.dot",
    feature_names=["petal length (cm)", "petal width (cm)"],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)

# Lenh terminal bien dich dot sang png:
# dot -Tpng iris_tree.dot -o iris_tree.png
```

2.3 Trực quan hóa Cây Iris

- Nút trên cùng gọi là **Nút gốc (Root Node)**.
- Nút không có con gọi là **Nút lá (Leaf Node)**.
- Mỗi nút hiển thị: Điều kiện chia, số samples (mẫu), value (phân bố), và class (nhãn dự đoán).

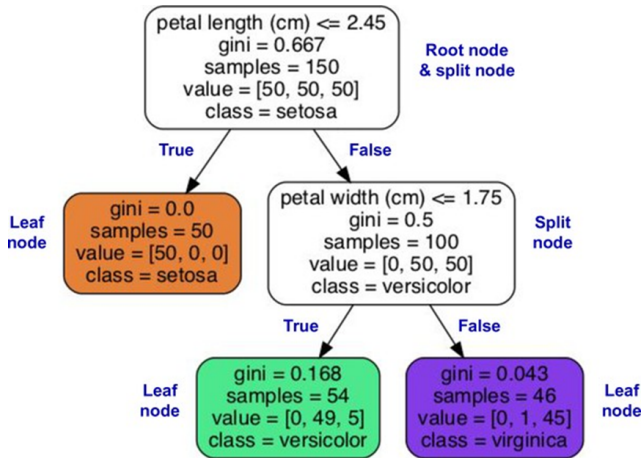


Figure: Hình 6-1: Cây Quyết định bộ dữ liệu Iris

3.1 Quá trình đưa ra dự đoán

- Cây sẽ duyệt từ **Nút gốc** đi xuống.
- Ở mỗi nút, nó kiểm tra một điều kiện (VD: $\text{petal_length} \leq 2.45 \text{ cm}$).
- Nếu **Đúng (True)**, đi sang nhánh bên Trái.
- Nếu **Sai (False)**, đi sang nhánh bên Phải.
- Lặp lại đến khi gặp Nút lá. Lớp có số lượng mẫu đông nhất trong nút lá đó sẽ là dự đoán cuối cùng.

3.2 Hiểu về Gini Impurity (Độ vẩn đục Gini)

- Tham số gini ở mỗi nút đo lường **độ vẩn đục** (impurity) của nó.
- Một nút được gọi là "thuần khiết" (pure) nếu $gini = 0$, nghĩa là mọi mẫu trong nút đó đều thuộc về **cùng một lớp**.
- Công thức Gini Impurity cho nút i :

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

- Trong đó: $p_{i,k}$ là tỷ lệ phần trăm mẫu thuộc lớp k nằm trong nút i .

3.3 Ví dụ tính toán Gini Impurity

- Xem lại Hình 6-1, Nút màu xanh lục (phía dưới bên trái):
- Có tổng cộng 54 mẫu. Lớp Setosa (0), Lớp Versicolor (49), Lớp Virginica (5).
- Tỷ lệ p : $p_{versicolor} = 49/54$, $p_{virginica} = 5/54$.
- Áp dụng công thức:

$$G_i = 1 - \left(\left(\frac{0}{54} \right)^2 + \left(\frac{49}{54} \right)^2 + \left(\frac{5}{54} \right)^2 \right)$$

$$G_i \approx 1 - (0 + 0.823 + 0.008) \approx 0.168$$

3.4 Ranh giới Quyết định (Decision Boundaries)

- Hình ảnh minh họa không gian đặc trưng.
- Đường nét đậm ở 2.45 cm (Trục tung) là ranh giới từ nút gốc.
- Đường đứt nét ở 1.75 cm (Trục hoành) là ranh giới chia nhánh thứ hai.
- Cây luôn tạo ra các đường ranh giới **vuông góc với trục tọa độ**.

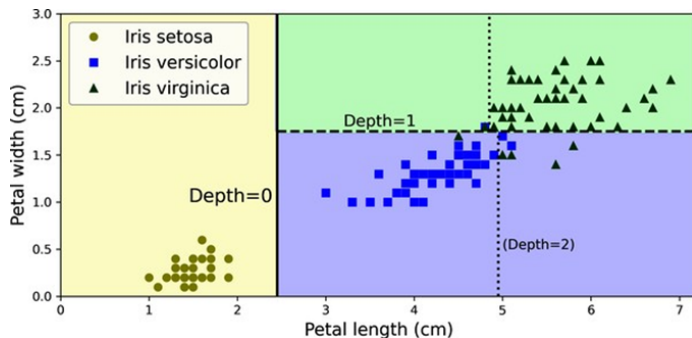


Figure: Hình 6-2: Ranh giới quyết định của Cây

3.5 Không cần Chuẩn hóa (Scaling) dữ liệu!

- Một ưu điểm tuyệt vời khác của Cây Quyết định: **Nó gần như không yêu cầu tiền xử lý dữ liệu!**
- Không cần StandardScaler hay MinMaxScaler.
- Việc một đặc trưng chạy từ 0-1 còn đặc trưng khác chạy từ 0-1000 không hề ảnh hưởng đến việc phân chia nhánh. Thuật toán chỉ tìm điểm ngưỡng cắt t_k .

3.6 Ước tính Xác suất Lớp (Class Probabilities)

- Cây quyết định cũng có thể trả về **xác suất** mà một mẫu thuộc về một lớp cụ thể.
- Bằng cách duyệt từ gốc xuống một nút lá.
- Xác suất của lớp k chính là tỉ lệ mẫu của lớp k trên tổng số mẫu tại nút lá đó.
- VD: Bông hoa dài 5cm, rộng 1.5cm sẽ đi vào nút lá màu xanh lục (49 Versicolor, 5 Virginica). Xác suất là $49/54 \approx 90.7\%$ cho Versicolor.

3.7 Code: Dự đoán Xác suất

```
# Dự đoán xác suất cho 1 bông hoa:
# petal_length = 5.0, petal_width = 1.5
probabilities = tree_clf.predict_proba([[5, 1.5]])
print("Xác suất (Setosa, Versi, Virgi):", probabilities)
# Output: [[0.          0.90740741 0.09259259]]

# Dự đoán lớp (argmax của xác suất)
pred_class = tree_clf.predict([[5, 1.5]])
print("Lớp dự đoán:", pred_class)
# Output: [1] (tức là Versicolor)
```

4.1 Thuật toán CART là gì?

- Scikit-Learn sử dụng thuật toán **CART** (Classification and Regression Tree) để huấn luyện cây.
- Thuật toán này khá đơn giản:
 - ➊ Phân chia tập huấn luyện thành 2 tập con sử dụng 1 đặc trưng k và ngưỡng t_k .
 - ➋ Chọn k và t_k sao cho tạo ra 2 tập con thuần khiết nhất (tức là Gini Impurity nhỏ nhất có thể).
 - ➌ Lặp lại đệ quy với từng tập con.
 - ➍ Dừng lại khi đạt đến `max_depth` hoặc không thể giảm thêm độ vẩn đục.

4.2 Hàm Chi phí của CART (Phân loại)

- Thuật toán CART sẽ tối thiểu hóa hàm chi phí (Cost Function) tại mỗi bước chia:

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

- Trong đó:
 - G_{left}, G_{right} là độ vẩn đục Gini của tập con bên trái/phải.
 - m_{left}, m_{right} là số lượng mẫu của tập con.
 - m là tổng số lượng mẫu tại nút đang xét.
- CART là thuật toán **Tham lam (Greedy Algorithm)**. Nó chỉ tìm kiếm mức phân tách tốt nhất tại *hiện tại*, không nhìn về tương lai. Do đó không bảo đảm tối ưu cục bộ.

4.3 Gini Impurity hay Information Entropy?

- Mặc định, CART dùng **Gini Impurity** (criterion="gini").
- Tuy nhiên, ta có thể đổi sang dùng **Entropy** (criterion="entropy").
- Entropy bằng 0 khi tập hợp hoàn toàn thuần khiết (chỉ chứa 1 lớp).
- Công thức Shannon's Entropy:

$$H_i = - \sum_{k=1, p_{i,k} \neq 0}^n p_{i,k} \log_2(p_{i,k})$$

4.4 So sánh Gini và Entropy

- Vậy nên sử dụng cái nào?
- Hầu hết mọi trường hợp, **Gini và Entropy tạo ra những cây rất giống nhau.**
- Gini tính toán **nhANH hơn một chút** (vì không chứa phép Logarit), là lựa chọn mặc định tốt.
- Entropy đôi khi có xu hướng tạo ra các cây cân bằng hơn (balanced trees) trong khi Gini thường cắt ra nhánh lớn nhất.

4.5 Độ phức tạp Thời gian (Time Complexity)

- **Quá trình Suy luận (Dự đoán):**

- Phải duyệt từ gốc đến lá. Độ sâu là $O(\log_2(m))$.
- Phức tạp dự đoán cực nhanh: $O(\log_2(m))$ độc lập với số lượng đặc trưng!

- **Quá trình Huấn luyện:**

- Thuật toán so sánh mọi đặc trưng (hoặc `max_features` nếu được cài) trên mọi ngưỡng.
- Phức tạp thời gian là $O(n \times m \log_2(m))$.
- Rất chậm đối với các tập dữ liệu có m lớn (Hàng triệu dòng).

5.1 Tại sao phải Chính quy hóa?

- Cây quyết định là mô hình **Non-parametric** (Phi tham số).
- Nghĩa là số lượng tham số không được ấn định trước, dẫn đến việc mô hình có thể phát triển tự do theo dữ liệu.
- Nếu không có giới hạn, cây sẽ cổ phân nhánh đến khi mỗi chiếc lá chỉ có 1 điểm duy nhất (Tương đương việc ghi nhớ học vẹt bộ dữ liệu).
- Điều này gây ra hiện tượng **Overfitting** cực kỳ nghiêm trọng.

5.2 Các Siêu tham số Chính quy hóa

- `max_depth`: Độ sâu tối đa của cây (Default: None). Giảm nó để hạn chế Overfitting.
- `min_samples_split`: Số lượng mẫu tối thiểu một nút phải có trước khi được phân chia.
- `min_samples_leaf`: Số mẫu tối thiểu một nút lá phải có.
- `max_leaf_nodes`: Số lượng nút lá tối đa.
- `max_features`: Số lượng đặc trưng tối đa được xem xét tại mỗi lần chia.

5.3 Hiệu ứng của Chính quy hóa

- Hình bên trái: Cây không giới hạn ($\text{min_samples_leaf}=1$). Ranh giới ngoằn ngoèo, Overfitting.
- Hình bên phải: Cây bị ràng buộc ($\text{min_samples_leaf}=4$). Ranh giới mượt mà hơn, khả năng tổng quát hóa (Generalization) tốt hơn trên dữ liệu mới.

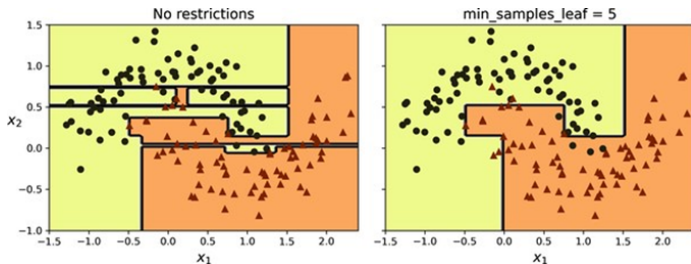


Figure: Hình 6-3: Cây không chính quy (trái) vs Chính quy hóa (phải)

6.1 Lớp DecisionTreeRegressor

- Cây quyết định cũng giải quyết xuất sắc các bài toán Hồi quy.
- Lúc này, thay vì dự đoán một "nhãn lớp", nút lá sẽ dự đoán một "giá trị liên tục" (Trung bình cộng của tất cả các mẫu nằm trong lá đó).
- Sử dụng hàm mục tiêu là MSE (Mean Squared Error) để tối ưu.

6.2 Code: Hồi quy dữ liệu Quadratic

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Tao du lieu mo phong y = x^2 + nhieu
np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

# Huan luyen cay hoi quy (Gioi han max_depth)
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

6.3 Trực quan hóa Cây Hồi quy

- Hình bên minh họa cây Hồi quy độ sâu 2.
- Ở mỗi nút lá, giá trị value chính là y trung bình của các mẫu, và mse (Mean Squared Error) là sai số tại nút đó.
- Dự đoán sẽ mang dạng các **bậc thang (step-wise)** vì trong cùng 1 khoảng chia, mọi giá trị x đều trả ra cùng 1 mức y.

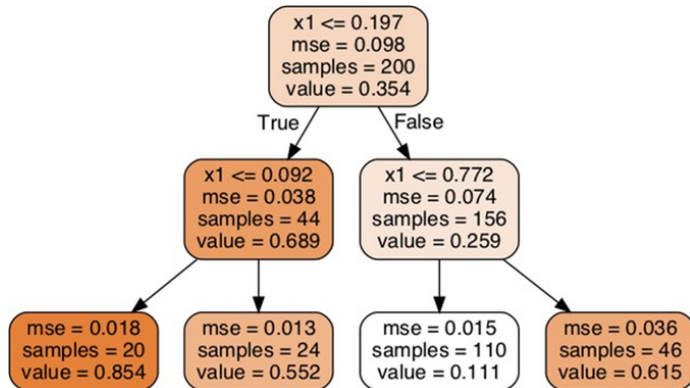


Figure: Hình 6-4: Sơ đồ Decision Tree Regression

6.4 Các Ranh giới Hồi quy (Bậc thang)

- Trái: $\text{max_depth} = 2$ (Có 4 bậc thang).
- Phải: $\text{max_depth} = 3$ (Có 8 bậc thang).
- Cây phân chia không gian thành các mảng và dự đoán y là đường thẳng hằng số trên từng mảng.

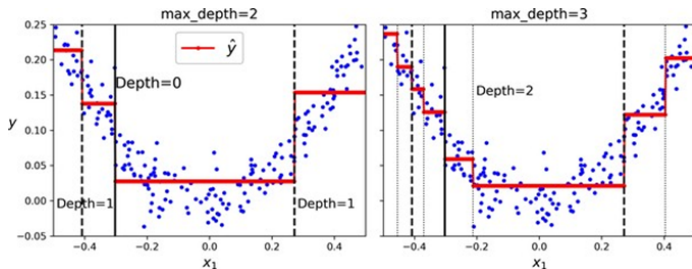


Figure: Hình 6-5: Hàm dự đoán của Cây Hồi quy

6.5 Overfitting trong Hồi quy

- Tương tự như phân loại, Cây Hồi quy rất dễ bị Overfitting nếu không ràng buộc.
- Cây sẽ uốn éo đứt đoạn để đi qua toàn bộ các nhiễu dữ liệu.
- Khắc phục: Dùng `min_samples_leaf=10` để ép mô hình mượt mà hơn.

6.6 Hàm Chi phí CART cho Hồi quy

- Thuật toán CART sẽ tối thiểu hóa sai số MSE thay vì Gini Impurity:

$$J(k, t_k) = \frac{m_{left}}{m} \text{MSE}_{left} + \frac{m_{right}}{m} \text{MSE}_{right}$$

- Trong đó:

$$\text{MSE}_{node} = \frac{1}{m_{node}} \sum_{i \in node} (\hat{y}_{node} - y^{(i)})^2$$

- Và $\hat{y}_{node}$ luôn là Trung bình cộng của các mẫu trong lá.

7.1 Trục chia vuông góc (Orthogonal splits)

- Cây quyết định yêu thích sự đơn giản. Nó luôn phân chia tập dữ liệu bằng các đường **song song hoặc vuông góc** với trục tọa độ.
- Nó không thể vẽ đường chéo hay đường cong!
- Điều này dẫn đến sự nhạy cảm cực độ đối với **Hướng Xoay của Dữ liệu (Axis Orientation)**.

7.2 Nhạy cảm với Xoay trục tọa độ

- Hình Trái: Dữ liệu phân tách tuyệt đẹp bằng 1 đường thẳng ngang. (Cây sâu 1 tầng).
- Hình Phải: Cùng bộ dữ liệu nhưng xoay 45 độ. Cây buộc phải chia cắt thành vô vàn đường cầu thang ziczac dốc để xấp xỉ một đường chéo.
- Cây rườm rà, tổng quát hóa kém.

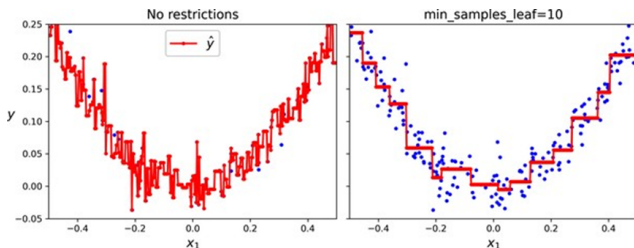


Figure: Hình 6-6: Độ nhạy cảm với trục xoay dữ liệu

7.3 Khắc phục Bằng PCA (Phân tích Thành phần Chính)

- Để giảm bớt sự nhạy cảm, ta có thể kết hợp Tiền xử lý bằng PCA (Principal Component Analysis).
- PCA sẽ xoay tập dữ liệu về trạng thái mà phương sai là lớn nhất trên các trục tọa độ.
- Nhờ đó, giới hạn khả năng cây bị bóp méo đường cầu thang.

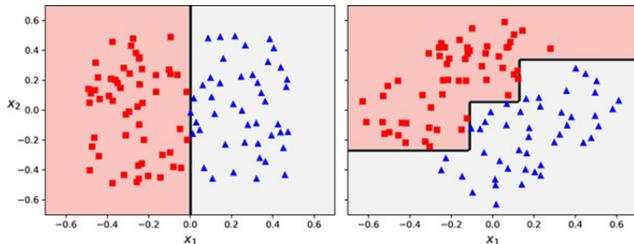


Figure: Hình 6-7: Cây phân loại sau khi xoay bằng PCA

7.4 Phương sai cao (High Variance)

- Vấn đề lớn nhất của Cây Quyết định: **Tính không ổn định cực cao (High Variance)**.
- Vì là một thuật toán tham lam (không kiểm tra lại toàn bộ tương lai), một **thay đổi vô cùng nhỏ** trong tập huấn luyện có thể tạo ra một cây **hoàn toàn khác biệt**.
- Một nhiễu loạn ngẫu nhiên nhỏ xíu ở nút gốc sẽ làm toàn bộ các nút con thay đổi.

7.5 Ví dụ về High Variance

- Trong bộ dữ liệu Iris, giả sử ta xóa **đúng 1 bông hoa** Versicolor to nhất (Dài 4.8cm, rộng 1.8cm).
- Cây mới học được (Bên phải) đã chia cắt dữ liệu theo ranh giới khác hoàn toàn so với cây ban đầu (Hình 6-2).

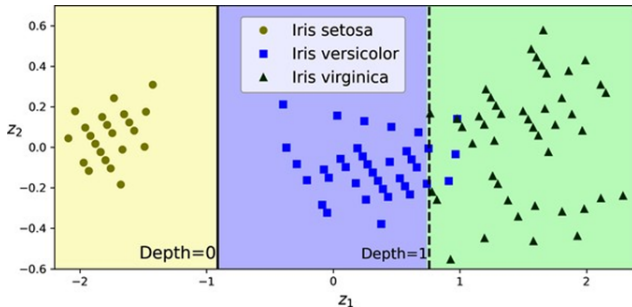


Figure: Hình 6-8: Cây bị méo mó chỉ sau khi xóa 1 điểm

7.6 Ngẫu nhiên tính trong Huấn luyện

- Ngay cả khi bạn **không xóa** bất kỳ dữ liệu nào, bản thân Scikit-Learn cũng chọn đặc trưng theo cách **ngẫu nhiên** để tăng tốc độ phân chia.
- Nghĩa là, huấn luyện cùng 1 tập dữ liệu 2 lần có thể ra 2 kết quả khác nhau (Nếu không set `random_state=42`).
- Sự ngẫu nhiên và Phương sai cao này thoạt nghe có vẻ là điểm yếu chí mạng...

7.7 Từ Điểm yếu trở thành Siêu Vị Thần (Random Forest)

- Thay vì dùng 1 cái cây không ổn định, ta sẽ trồng hàng nghìn cái cây như thế!
- Bằng cách trung bình hóa dự đoán của một **Khu Rừng (Forest)** các cây có phương sai cao và độc lập nhau.
- Kết quả là thuật toán **Random Forest** - một trong những thuật toán ML vĩ đại nhất lịch sử ra đời!

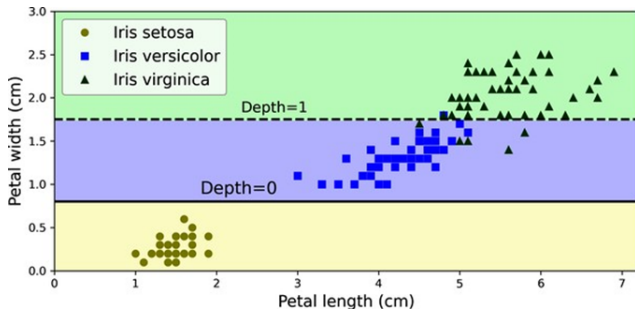


Figure: Hình 6-9: Khu rừng Random Forest (Chương 7)

8.1 Ưu điểm tóm tắt

- **Dễ hiểu, Dễ giải thích:** Có thể xuất ra dạng quy tắc (If-Else) hoặc sơ đồ luồng.
- **Không yêu cầu Chuẩn hóa:** Hoạt động tốt ngay cả khi các biến khác đơn vị, hoặc lẫn lộn chữ và số.
- **Xử lý Phi tuyến tính:** Cắt xẻ không gian tọa độ cực tốt, khớp với ranh giới phức tạp.
- **Nhanh ở khâu Dự đoán:** Suy luận với tốc độ Logarit $O(\log_2 m)$.

8.2 Nhược điểm tóm tắt

- **Overfitting nặng nề:** Thích ôm chặt lấy dữ liệu huấn luyện nếu không bị kìm hãm bằng Regularization.
- **Nhạy cảm với Xoay trục:** Bị trói buộc vào các ranh giới song song trục tọa độ (Khắc phục bằng PCA).
- **Instability (Bất ổn / Variance Cao):** Rung lắc mạnh khi dữ liệu chỉ đổi một tẹo. (Khắc phục bằng Ensemble Learning).

8.3 Bài tập 1: Phân loại Moons

- Khởi tạo bộ dữ liệu 10.000 điểm bằng `make_moons(n_samples=10000, noise=0.4)`.
- Phân chia thành Train/Test bằng `train_test_split`.
- Dùng `GridSearchCV` với `DecisionTreeClassifier` để dò tìm siêu tham số `max_leaf_nodes` tốt nhất.
- Huấn luyện và tính Accuracy Score.

8.4 Bài tập 2: Tự xây dựng Khu rừng (Forest)

- Bổ sung từ Bài tập 1. Thay vì 1 cây, hãy tạo 1000 tập con dữ liệu từ Train set.
- Dùng vòng lặp huấn luyện 1000 Cây Quyết Định (với cấu hình tốt nhất từ Bài 1) trên mỗi tập con.
- Dự đoán trên Test set: Bằng cách áp dụng **Mode** (Lấy dự đoán được số đông vote nhiều nhất) từ 1000 cây cho mỗi mẫu.
- Độ chính xác của bạn sẽ tăng hơn ít nhất 0.5 – 1.5% so với 1 cây.

HỎI & ĐÁP